

JUPITER - SECURITY REVIEW

Jupiter Offerbook.

A security review of Jupiter Offerbook.

CLIENT

Jupiter

PROJECT

Offerbook

AUDITORS

Nicholas Chew,
Jakub Zmyslowski,
Minato Namikazi

AUDIT WINDOW

June 22 - July 24,
2026

FINAL REPORT

July 31, 2026

§ 00

Contents.

52 pages - overview, confidence ranking, scope, 36 findings, disclaimer, and about Guardian.

§ 01	Overview	p. 03
§ 02	Confidence Ranking	p. 04
§ 03	Scope & Classifications	p. 05
	Files in Scope	p. 06
§ 04	Findings	p. 07
	Round 1 - Main Review • 30 findings	p. 11-44
	Round 2 - Remediation Review • 6 findings	p. 45-50
§ 05	Disclaimer	p. 51
§ 06	About Guardian	p. 52

Overview.

Guardian reviewed Jupiter Offerbook from June 22 through July 8, 2026, followed by a remediation review from July 18 through July 24. The final report includes 4 Medium findings, 15 Low findings, 17 Info findings.

AUDIT FIRM	Guardian	REPOSITORY	jup-ag/offerbook-program
CLIENT	Jupiter	FEATURE REVIEW	jup-ag/offerbook-program#7
PROJECT	Offerbook	MAIN REVIEW COMMIT	34e7b2889ed037ebed2cad878347ee0255fc257a
PREPARED BY	Nicholas Chew, Jakub Zmyslowski, Minato Namikazi	REMEDIATION COMMIT	bed9f6e5d5bc3377648fb0357db0c667fc55e0ce
AUDIT WINDOW	June 22 - July 24, 2026		
FINAL REPORT	July 31, 2026		

1.1 Confidence Ranking

4/5

Guardian assigns a Confidence Ranking of 4 out of 5. No High or Critical findings were identified. The engagement found four Medium-severity issues across the main and remediation reviews, while the remaining findings were Low or Informational; identified items were either remediated or acknowledged.

High Confidence

1 V. LOW	2 LOW	3 MED	4 HIGH	5 V. HIGH
-------------	----------	----------	-----------	--------------

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	IN PROG	DECLINED	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0	0	0
High	0	0	0	0	0	0	0
Medium	4	0	0	0	2	0	2
Low	15	0	0	0	9	0	6
Info	17	0	0	0	10	0	7

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3 Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas such as weak modularity or risky patterns. No critical design flaws were identified, though some patterns could lead to edge-case issues. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2 Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week, or additional unresolved High/Critical findings uncovered in remediation review.
1 Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5 /week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

§ 03

Scope & Classifications.

Guardian reviewed the Jupiter Offerbook Solana program in `jup-ag/offerbook-program`, including fungible and non-fungible offer creation and fills, escrow handling, repayment, default claims, configuration, fees, events, and asset validation. The exported scope contains 6,691 source lines across 43 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MEDIUM	IMP · LOW	IMPACT · HIGH	Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.
LIK · HIGH	Critical	High	Medium	IMPACT · MEDIUM	Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.
LIK · MED	High	Medium	Low	IMPACT · LOW	Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.
LIK · LOW	Medium	Low	Low	LIKELIHOOD · HIGH	Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.
				LIKELIHOOD · MEDIUM	Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.
				LIKELIHOOD · LOW	Unlikely to ever occur in production.

Scope.

Files in scope, grouped by repository. SLOC counts are physical source lines.

jup-ag/offerbook-program (1/2)		22 FILES · 5871 SLOC	jup-ag/offerbook-program (2/2)		21 FILES · 820 SLOC
programs/offerbook/src/instructions/non_fungible			programs/offerbook/src/instructions/non_fungible/classic		
fill_non_fungible_principal_offer.rs	742		escrow_classic_nft_withdraw.rs	86	
fill_non_fungible_collateral_offer.rs	606		mod.rs	4	
repay_non_fungible_loan.rs	545		programs/offerbook/src/instructions		
claim_non_fungible_loan.rs	396		update_config.rs	71	
create_non_fungible_principal_offer.rs	176		claim_fee.rs	58	
create_non_fungible_collateral_offer.rs	153		create_user.rs	55	
programs/offerbook/src/instructions/fungible			init.rs	38	
fill_token_principal_offer.rs	366		cancel_offer.rs	25	
fill_token_collateral_offer.rs	361		mod.rs	14	
repay_token_loan.rs	268		programs/offerbook/src/instructions/non_fungible/mpl_core		
claim_token_loan.rs	224		escrow_core_nft_withdraw.rs	70	
create_token_principal_offer.rs	177		escrow_core_nft_deposit.rs	68	
create_token_collateral_offer.rs	176		mod.rs	4	
programs/offerbook/src			programs/offerbook/src/instructions/fungible		
events.rs	351		escrow_token_deposit.rs	68	
utils.rs	189		escrow_token_withdraw.rs	67	
lib.rs	167		mod.rs	16	
programs/offerbook/src/state			programs/offerbook/src/state		
offer.rs	204		user.rs	68	
config.rs	175		mod.rs	10	
loan.rs	108		programs/offerbook/src		
asset.rs	93		error.rs	46	
programs/offerbook/src/instructions/non_fungible/programmable			constants.rs	3	
escrow_programmable_nft_withdraw.rs	148		programs/offerbook		
escrow_programmable_nft_deposit.rs	145		Cargo.toml	27	
programs/offerbook/src/instructions/non_fungible/classic			programs/offerbook/src/instructions/non_fungible		
escrow_classic_nft_deposit.rs	101		mod.rs	18	
			programs/offerbook/src/instructions/non_fungible/programmable		
			mod.rs	4	

§ 04

Findings.

36 findings across two rounds: 4 Medium findings, 15 Low findings, 17 Info findings. 15 were resolved and 21 were acknowledged. Visible IDs reset by severity within each round.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
M-01	Token Delegates Can Block Collateral Transfer	Warning	MEDIUM	ACKNOWLEDGED	p. 11
M-02	Token-2022 mint mutability bypass	Logical Error	MEDIUM	RESOLVED	p. 12
M-03	Unsafe Token-2022 NFT Collateral	Logical Error	MEDIUM	RESOLVED	p. 13
L-01	Referral Rewards Are Only Events	Gaming	LOW	ACKNOWLEDGED	p. 15
L-02	Single-Step Admin Transfer	Access Control	LOW	ACKNOWLEDGED	p. 16
L-03	Fee Math Can Block Claims	Math	LOW	ACKNOWLEDGED	p. 17
L-04	Counter offers can be sniped	Frontrunning	LOW	ACKNOWLEDGED	p. 18
L-05	Partial fill rounding undercollateralizes loans	Rounding	LOW	ACKNOWLEDGED	p. 19
L-06	Core collection filter skips membership check	Warning	LOW	RESOLVED	p. 21
L-07	FVC filter trusts creator key	Warning	LOW	ACKNOWLEDGED	p. 22
L-08	Classic NFT claim destination is UI-trusted	Validation	LOW	RESOLVED	p. 23

§ 04

Findings, continued.

Index page 2 of 4.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
L-09	Classic NFT Withdraw Uses Readonly ATA	Unexpected Behavior	LOW	RESOLVED	p. 24
L-10	Loan interest can round down to zero	Rounding	LOW	ACKNOWLEDGED	p. 25
L-11	Auth rule pNFTs can brick escrow withdrawals	Logical Error	LOW	RESOLVED	p. 27
L-12	NFT misclassified as programmable deposit	Validation	LOW	RESOLVED	p. 28
L-13	Fee exemption missing for RWA collateral	Logical Error	LOW	ACKNOWLEDGED	p. 30
I-01	Unbounded Max APY Configuration	Validation	INFO	RESOLVED	p. 31
I-02	Config Updates Emit No Events	Events	INFO	ACKNOWLEDGED	p. 32
I-03	Offers Are Not Balance Backed	Unexpected Behavior	INFO	ACKNOWLEDGED	p. 33
I-04	Docs deviate from market logic	Documentation	INFO	ACKNOWLEDGED	p. 34
I-05	Counter offer UI can confuse NFT collateral	Warning	INFO	RESOLVED	p. 35
I-06	Counters can point to expired offers	Unexpected Behavior	INFO	ACKNOWLEDGED	p. 36

§ 04

Findings, continued.

Index page 3 of 4.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 1 - MAIN REVIEW					
I-07	NFT claim event implies escrow custody	Events	INFO	RESOLVED	p. 37
I-08	Active Loan Fees Are Mutable	Unexpected Behavior	INFO	ACKNOWLEDGED	p. 38
I-09	Config PDA Not Enforced	Best Practices	INFO	ACKNOWLEDGED	p. 39
I-10	Dead Collection Attribute Filter	Superfluous Code	INFO	ACKNOWLEDGED	p. 40
I-11	Classic NFT Loans Strand Empty ATAs	Unexpected Behavior	INFO	RESOLVED	p. 41
I-12	Config min hikes strand live offers	Warning	INFO	ACKNOWLEDGED	p. 42
I-13	Misdocumented Repay Fee	Documentation	INFO	RESOLVED	p. 43
I-14	Undocumented LoanType Field	Documentation	INFO	ACKNOWLEDGED	p. 44
ROUND 2 - REMEDIATION REVIEW					
M-01	Closing escrow blocks classic NFT repayment	DoS	MEDIUM	ACKNOWLEDGED	p. 45
L-01	Extensions bypass the emergency pause	Warning	LOW	ACKNOWLEDGED	p. 46
L-02	Extension emits zero-amount referral rewards	Rounding	LOW	RESOLVED	p. 47

§ 04

Findings, continued.

Index page 4 of 4.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
ROUND 2 - REMEDIATION REVIEW					
I-01	Offer events omit extendability	Events	INFO	ACKNOWLEDGED	p. 48
I-02	Token-2022 mutability remains	Warning	INFO	RESOLVED	p. 49
I-03	Filtered counter collateral can mismatch	Warning	INFO	RESOLVED	p. 50

MEDIUM **M-01**

ACKNOWLEDGED

Token Delegates Can Block Collateral Transfer.

CATEGORY Warning LOCATION `utils.rs:18` REVIEW Round 1 - Main Review

DESCRIPTION

Token offer creation validates Token-2022 mints through `validate_mint_extensions()`, but that helper only rejects nonzero transfer fees and configured transfer hooks. It does not reject the `PermanentDelegate` extension, and it also allows collateral mints with other external control surfaces such as a configured freeze authority or the Token-2022 `Pausable` extension.

For token-collateral loans, collateral is moved into a loan vault whose token owner is the loan PDA. The protocol later assumes that only the loan PDA can move those tokens during repayment or default claim. With a Token-2022 collateral mint that has a permanent delegate, the mint-level delegate can transfer or burn tokens from any token account for that mint, including the loan vault, without the loan PDA signing.

Example: a lender accepts or creates an offer against Token-2022 collateral that has a permanent delegate controlled by the borrower or a cooperating party. After the collateral is locked in the loan vault, the delegate drains or burns the vault balance. The lender's later default claim can receive less collateral or fail while trying to transfer the liquidation fee.

RECOMMENDATION

Reject Token-2022 collateral mints with `PermanentDelegate` configured. Also reject mints with retained freeze authority or `Pausable` control unless the protocol explicitly intends to trust that authority.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged the risk. Supporting Token-2022 assets with a `PermanentDelegate` is intentional because legitimate assets, including PayPal USD, use this extension. Users who accept Token-2022 assets are expected to assess the associated authority and extension risks.

MEDIUM **M-02**

RESOLVED

Token-2022 mint mutability bypass.

CATEGORY Logical Error

LOCATION programs/offerbook/src/utils.rs#L18-L47

REVIEW Round 1 - Main Review

DESCRIPTION

Token-2022 mint extensions are validated only when an offer is created, but the program does not reject mutable extension authorities and does not revalidate the mint during fill, repayment, or claim flows. Because the authority fields are not checked, a Token-2022 mint that is safe at offer creation can later be updated to enable transfer fees or a transfer hook. This can cause loan collateral transfers, repayments, collateral returns, or default claims to transfer less than the protocol expects or to revert entirely.

RECOMMENDATION

Reject Token-2022 mints whose transfer-fee or transfer-hook authorities are set. Require `transfer_fee_config_authority` to be unset when `TransferFeeConfig` is present, and require `authority` to be unset when `TransferHook` is present.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <https://github.com/jup-ag/offerbook-program/pull/8/commits/30c1e56e7c0ba6eb2fc4c6999b5676fb566a534f>.

MEDIUM **M-03**

RESOLVED

Unsafe Token-2022 NFT Collateral.

CATEGORY Logical Error

LOCATION programs/offerbook/src/instructions/non_fungible/fill_non_fungible_collateral_offer.rs#L163-L168

REVIEW Round 1 - Main Review

DESCRIPTION

NFT collateral can be supplied across three standards (classic, programmable, and MPL Core). Broad `Collection` or `FirstVerifiedCreator` offers let the borrower select the concrete asset only at fill time. None of the fill paths validate that the chosen asset's transfer controls will still allow the loan PDA to move the collateral on a later default claim. This affects all three standards.

Classic NFTs (Token-2022 extensions)

Classic NFT flows accept NFT collateral mints through the generic token interface and only require the mint to look like an NFT by checking `supply == 1` and `decimals == 0`.

TEXT

```
1      #[account(  
2          constraint = nft_mint.supply == 1 && nft_mint.decimals == 0 @ OfferbookError::InvalidTokenMint,  
3          token::token_program = token_program,  
4      )]  
5      pub nft_mint: Box<InterfaceAccount<'info, Mint>>,
```

However, NFT collateral mints are not subject to any Token-2022 extension or mutable-authority validation before being accepted into a loan. As a result, a Token-2022 mint can be used as classic NFT collateral while retaining external transfer controls such as `PermanentDelegate`, freeze authority, pausable controls, transfer-hook authority, or transfer-fee authority. A malicious borrower can lock a Token-2022 NFT whose mint has a permanent delegate or mutable hook. Consequently, lenders may be unable to claim defaulted NFT collateral, or the collateral may be removed from the loan vault after being accepted.

Programmable NFTs (authorization rule sets)

Programmable NFT transfers are gated by Metaplex authorization rules. The fill path checks only the metadata collection or creator filter, then forwards the supplied `authorization_rules` account to the transfer CPI without verifying that the rule set will also permit the later loan-PDA-to-lender claim transfer. A borrower can fill a broad offer with a pNFT whose rule set allows the deposit into loan custody but rejects the claim transfer from the loan PDA to the lender. Because a rule set can be mutable, even a rule set that is permissive at fill time can later be changed by its authority after the pNFT is locked.

MEDIUM **M-03** CONTINUED 2/2

RESOLVED

Unsafe Token-2022 NFT Collateral.

DESCRIPTION

Core NFTs (transfer-affecting plugins)

MPL Core assets can carry plugins such as `FreezeDelegate`, `PermanentFreezeDelegate`, `TransferDelegate`, `PermanentTransferDelegate`, oracle plugins, and lifecycle hooks, and plugin authorities can be addresses outside the protocol. The fill path inspects only the collection account or the `VerifiedCreators` plugin, not transfer-affecting plugins. A borrower can fill a broad Core offer with an asset whose freeze or transfer plugin is controlled by an external authority, then freeze the asset or block transfers before default so the claim transfer fails even though the loan PDA owns the asset.

RECOMMENDATION

Validate NFT collateral before accepting it into a loan, for every standard:

- Classic: validate classic NFT collateral mints before accepting them into a loan. Reject Token-2022 NFT collateral mints with `PermanentDelegate`, retained freeze authority, pausable controls, transfer hooks, transfer-fee authorities, or other mutable transfer-control extensions.
- Programmable: require the rule set to be absent, or immutable and permissive for the loan-PDA-to-lender and loan-PDA-to-borrower transfer paths. A fill-time permissiveness check alone is insufficient because a mutable rule set can be changed after the pNFT is locked.
- Core: reject assets carrying `FreezeDelegate`, `PermanentFreezeDelegate`, `TransferDelegate`, `PermanentTransferDelegate`, oracle, or lifecycle-hook plugins, or require that any freeze or transfer-affecting plugin authority be the loan PDA so neither the borrower nor a third party can alter transferability after fill.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <http://github.com/jup-ag/offerbook-program/pull/8/commits/8148e85b09b432b0e23126cab4872b2a12b40bb1>.

LOW L-01

ACKNOWLEDGED

Referral Rewards Are Only Events.

CATEGORY Gaming LOCATION utils.rs:118 REVIEW Round 1 - Main Review

DESCRIPTION

In every fee-bearing path the full computed `protocol_fee` is transferred to the fee authority token account, while the referral and referee reward splits are only emitted as events and never moved on-chain.

Because the reward events are derived only from on-chain activity and self-dealing is not prevented across separate wallets, a user can manufacture reward events. The offer-creator check only blocks `offer.creator == signer`, so the same actor using two wallets (one registered with a referrer) can repeatedly fill and repay loans between the wallets to emit `ReferralReward` events crediting a colluding referrer.

RECOMMENDATION

Treat referral and referee reward events as untrusted hints. Settle rewards offchain using sybil and wash trade resistant logic rather than raw event amounts, or move reward accounting and payout onchain with explicit anti self-dealing constraints.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged the issue. Referral rewards are funded from user-paid protocol fees, so Sybil activity cannot create a protocol loss. The team accepts that users may obtain a small fee rebate through this behavior.

LOW L-02

ACKNOWLEDGED

Single-Step Admin Transfer.

CATEGORY Access Control LOCATION programs/offerbook/src/state/config.rs#L92-L101

REVIEW Round 1 - Main Review

DESCRIPTION

The admin role can be transferred in a single transaction without requiring the new admin to accept the role. The current logic only checks that the new address is not the current admin and is not the default pubkey, then immediately overwrites the admin field.

TEXT

```
1 pub fn set_admin(&mut self, pubkey: &Pubkey) → Result<()> {
2     require!(!self.is_admin(pubkey), OfferbookError::NewAdminIsOldAdmin);
3     require!(
4         !pubkey.eq(&Pubkey::default()),
5         OfferbookError::NotAuthorized
6     );
7
8     self.admin = *pubkey;
9     Ok(())
10 }
```

If the current admin mistakenly submits an incorrect address administrative control may be permanently lost. Since the admin controls protocol configuration such as pausing, repayment settings, fee parameters, and risk limits, this can leave the protocol unable to react to emergencies or restore normal operation.

RECOMMENDATION

Consider implementing a two-step admin transfer flow. The current admin should nominate a pending admin, and only that nominated address should be able to separately accept this role.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged the risk and considers program upgradeability, including hardcoding a replacement administrator if necessary, an adequate recovery mechanism.

LOW **L-03**

ACKNOWLEDGED

Fee Math Can Block Claims.

CATEGORY Math

LOCATION `programs/offerbook/src/utils.rs#L80-L82`

REVIEW Round 1 - Main Review

DESCRIPTION

The `compute_protocol_fee` helper performs multiplication in u64 before dividing by `10_000`. This helper is reused for liquidation fees during token collateral default claims, where the fee base is the full collateral amount.

TEXT

```
1 rust
2 fn compute_protocol_fee(interest: u64, protocol_fee_bps: u32) → u64 {
3     interest * u64::from(protocol_fee_bps) / 10_000
4 }
```

The program is compiled with overflow checks enabled, so if `loan.collateral_amount * liquidation_fee_bps` exceeds `u64::MAX`, the default claim aborts before collateral is transferred. Offer creation only checks that collateral is above the configured minimum, but it does not enforce an upper bound that keeps fee multiplication within range. This is especially relevant because the program does not implement an on-chain collateral whitelist. Any supported token mint can be used as collateral if the offer creator accepts it, including long-tail assets with very large supplies, high decimals, or unusual denominations. A borrower can therefore propose a loan using a high amount long-tail collateral token that makes the liquidation fee calculation overflow. If the loan defaults, the lender's claim can consistently fail during fee calculation, leaving collateral stuck in the loan vault.

RECOMMENDATION

Perform fee calculations in u128 and only downcast after division.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged the issue and considers an interest value large enough to overflow the multiplication highly improbable in practice.

LOW L-04

ACKNOWLEDGED

Counter offers can be sniped.

CATEGORY Frontrunning

LOCATION fill_token_principal_offer.rs, fill_token_collateral_offer.rs, fill_non_fungible_principal_offer, fill_non_fungible_collateral_offer.rs

REVIEW Round 1 - Main Review

DESCRIPTION

Counter offers are not restricted to the creator of the offer being countered. When a user creates a counter offer, the program stores the referenced offer in `countered_offer`, but later fill instructions do not require the filler to be the creator of that referenced offer. They only require the selected counter offer to be active, unexpired, and not filled by its own creator.

This means a third party can fill a counter offer before the intended original offer owner accepts it. For example, borrower A posts an offer, lender B creates a more attractive counter offer that points to A's offer, and borrower C sees B's counter in the mempool or indexer. If C can satisfy B's collateral terms, C can fill B's counter offer first. B receives the exact published loan terms, but A loses the intended counter-offer opportunity.

The root cause is that `countered_offer` is only a matching hint for clients and indexers. It is not used as an on-chain authorization target during fill.

RECOMMENDATION

If counter offers are intended to be exclusive to the referenced offer creator, add a dedicated accept-counter path or fill constraint that requires the filler to match the creator of the referenced `countered_offer`. That path should also define whether the referenced original offer and sibling counters are consumed, cancelled, or left live.

If counter offers are intended to be public offers with a UI link, update client and documentation language to make that explicit and avoid presenting them as exclusive to the original offer owner.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged the potential confusion. All offers are intentionally public and may be accepted by any user; `countered_offer` is only an off-chain association used to display related offers.

LOW L-05

ACKNOWLEDGED

Partial fill rounding undercollateralizes loans.

CATEGORY Rounding LOCATION fill_token_principal_offer.rs:218, fill_token_collateral_offer.rs:220

REVIEW Round 1 - Main Review

DESCRIPTION

For partial-fillable fungible offers, the paired-side amount is computed with floor (integer) division against the offer's original totals, and `Offer::take_loan` marks the offer `Fulfilled` as soon as either side reaches zero. Each fill therefore drops up to one raw unit of the paired asset, and the abandoned remainder is never collected.

On a principal offer the borrower posts the pro-rata collateral:

RUST

```
1 // fill_token_principal_offer.rs::compute_collateral_amount
2 (u128::from(offer.collateral_amount) * u128::from(self.principal_fill_amount)
3   / u128::from(offer.principal_amount))
4 .try_into()?
```

The symmetric principal computation on the collateral-offer path has the same shape. The floor means `sum(floor(collateral_amount * pi / principal_amount)) ≤ collateral_amount`, so once all principal is drawn the lender has received slightly less collateral than the advertised ratio demanded. The dropped fractions are made permanent because the offer closes when the principal side hits zero, leaving the leftover `remaining_collateral` capacity unenforced.

Impact

LOW **L-05** CONTINUED 2/2

ACKNOWLEDGED

Partial fill rounding undercollateralizes loans.

DESCRIPTION

Under default configuration this is dust. Each fill must lock more than `min_collateral_amount` (and more than `min_principal_amount` on the other path), which bounds the total relative undercollateralization to `1 / min_collateral_amount`.

With the default `min_collateral_amount = 10_000` the worst case is `< 0.01%`, which is economically negligible. The issue only becomes material if an admin lowers `min_collateral_amount` (or `min_principal_amount`) toward single digits, which simultaneously defeats the dust guard those parameters exist to provide. `set_min_collateral_amount` only enforces `> 0`, so a misconfiguration to `1` or `2` is permitted and would allow the relative loss to approach significant levels.

RECOMMENDATION

This is defense-in-depth; no change is required under sane configuration.

- Keep `min_principal_amount` and `min_collateral_amount` set high in the collateral token's real decimals in production
- Consider enforcing a sane lower bound in `set_min_collateral_amount / set_min_principal_amount` beyond `> 0`.

RESOLUTION

Status: Acknowledged.

LOW **L-06**

RESOLVED

Core collection filter skips membership check.

CATEGORY Warning

LOCATION fill_non_fungible_principal_offer.rs:418

REVIEW Round 1 - Main Review

DESCRIPTION

NFT principal offers can use a broad `Collection` filter, and the concrete collateral is resolved at fill time. For MPL Core collateral, the filter check only compares the pubkey of the separately supplied `collection` account to the filter value. It never reads the locked asset's own collection membership from the `BaseAssetV1` account.

`get_collection` returns the key of the passed `collection` account, and `validate_filter` compares that key to `collection_filter.collection` :

RUST

```
1 fn get_collection(&self) → Result<Option<Pubkey>> {
2     Ok(self.collection.as_ref().map(|c| c.key()))
3 }
4 // AssetFilter::Collection(collection_filter) ⇒
5 require!(collection == Some(collection_filter.collection), OfferbookError::InvalidCollateral);
```

So the on-chain filter passes whenever the borrower supplies the real filtered collection account, regardless of whether the asset they are locking actually belongs to that collection. This differs from the other paths that bind to the asset being locked: classic and pNFT bind `nft_metadata` to the locked mint via `Metadata::find_pda` and require a `verified` collection, and the Core `FirstVerifiedCreator` branch reads the `VerifiedCreators` plugin directly off the asset.

What currently prevents a mismatch is external, not this check: `lock_nft` invokes the MPL Core `TransferV1` with both `asset` and `collection`, and MPL Core rejects the transfer if the asset's update authority is not that collection. The program's own filter validation is insufficient on its own and relies entirely on the lock CPI to enforce membership.

RECOMMENDATION

In the Core `Collection` branch of `validate_filter`, read the update authority from the `BaseAssetV1` asset account and require it equals `UpdateAuthority::Collection(collection_filter.collection)`, mirroring how the `FirstVerifiedCreator` branch reads the plugin off the asset.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <https://github.com/jup-ag/offerbook-program/pull/8/changes/639bae626af8ad9525eebef03602bf3e80e8f52f>.

LOW **L-07**

ACKNOWLEDGED

FVC filter trusts creator key.

CATEGORY Warning

LOCATION fill_non_fungible_principal_offer.rs:435

REVIEW Round 1 - Main Review

DESCRIPTION

Principal side NFT offers can be scoped with a `FirstVerifiedCreator` (FVC) filter. For these broad offers, creation stores `Asset::None` plus a creator key instead of a concrete collateral asset or collection. At fill time, the borrower chooses the concrete NFT, and the filter check only requires the NFT's first verified creator to match the stored creator key.

The Metaplex path compares `metadata_first_verified_creator(metadata)` to the filter creator. The MPL Core path performs the same kind of single-key comparison after reading the `VerifiedCreators` plugin from the asset.

This means an FVC offer trusts the named creator key as the collateral universe. A creator key can verify NFTs across multiple collections or future drops, and all of those assets can satisfy the same FVC offer even if only one collection is valuable. For example:

- Alice offers to lend 1000 USDC against any NFT whose first verified creator is Bob.
- Bob or another holder can fill the offer with a low value NFT from a different Bob-verified drop.
- If the loan defaults, Alice receives that low value NFT.

The risk is that FVC can be mistaken for collection-level collateral selection, while it actually relies on the named creator not to verify low-value sibling assets.

RECOMMENDATION

Disable FVC for value sensitive lending, or clearly label it as creator-trusted collateral selection. For stronger filtering, require broad NFT offers to use a verified collection, a protocol or lender allowlist of acceptable collections, or a combined "collection plus creator" filter.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged the issue. The first-verified-creator filter remains necessary to support legacy NFT collections created before the Metaplex collection standard, where the first verified creator identifies the collection.

LOW L-08

RESOLVED

Classic NFT claim destination is UI-trusted.

CATEGORY Validation

LOCATION `claim_non_fungible_loan.rs:89`

REVIEW Round 1 - Main Review

DESCRIPTION

The classic NFT default-claim path accepts `lender_collateral_escrow` as a remaining account and only constrains it to be a token account for the NFT mint and token program. It does not require the destination token account to be owned by the lender, the lender's user PDA, or the lender's associated token account.

The transfer then moves the claimed NFT from the loan vault to that supplied token account, and the emitted `EscrowDeposit` event records `user: signer`. For the happy path, tests pass the lender's ATA. However, a wallet, SDK, or route builder that supplies a different token account for the same mint can direct the classic NFT collateral somewhere else while the transaction still looks like the lender's claim.

The other paths are not exposed to this. The fungible claim path is stricter and pins the destination to the lender user's escrow ATA via `associated_token::authority = signer_user`. Core NFT claims set `new_owner` directly to the lender signer and use no token account, so there is no client-supplied destination to redirect. Programmable NFT claims do pass an equally loose `lender_nft_token_account` (constrained only by mint and token program), but the Token Metadata `TransferV1` CPI fixes `destination_owner` to the lender signer and binds the destination token account to that owner's associated token account, so the supplied account cannot point elsewhere. The classic path is the lone outlier because it performs a raw SPL `transfer_checked` to the supplied account with no owner binding.

RECOMMENDATION

Require the classic NFT claim destination to be the lender's canonical ATA, or at least require `token::authority = signer` on the destination token account. If arbitrary destinations are intentionally supported, client builders should always derive and display the exact destination owner and token account before signature.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <https://github.com/jup-ag/offerbook-program/pull/8/changes/1f68860c32c29dc9cdfa7f514815d936cb026897>.

LOW L-09

RESOLVED

Classic NFT Withdraw Uses Readonly ATA.

CATEGORY Unexpected Behavior

LOCATION programs/offerbook/src/instructions/non_fungible/classic/escrow_classic_nft_withdraw.rs#L18-L23

REVIEW Round 1 - Main Review

DESCRIPTION

The classic NFT withdraw instruction transfers the NFT from the user escrow token account back to the signer's NFT token account, but the destination token account is not marked as mutable in the account context.

TEXT

```
1      #[account(  
2          token::mint = nft_mint,  
3          token::authority = signer,  
4          token::token_program = token_program,  
5      )]  
6      pub signer_nft_token_account: Box<InterfaceAccount<'info, TokenAccount>>,
```

Because the account is missing `mut`, the generated IDL exposes the signer NFT token account as readonly, even though its balance must be updated to receive the NFT. Standard IDL-driven clients that call the withdraw instruction directly will therefore submit a readonly destination account, while the SPL Token transfer requires that destination account to be writable.

The transaction fails with a writable privilege escalation error, blocking public IDL users from withdrawing escrowed classic NFTs through the standalone withdraw instruction. Success currently depends on the client adding a separate instruction that makes the destination token account writable at the transaction level.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <https://github.com/jup-ag/offerbook-program/pull/8/changes/167475d6f70efb82a45fb4ba6bf6cb6eae24af6f>.

LOW L-10

ACKNOWLEDGED

Loan interest can round down to zero.

CATEGORY Rounding

LOCATION programs/offerbook/src/utils.rs

REVIEW Round 1 - Main Review

DESCRIPTION

`compute_interest_amount` computes interest with a single truncating integer division and no minimum-interest floor:

RUST

```
1 let interest_denominator: u128 = 10_000 * 31_536_000; // 315_360_000_000
2 let interest: u128 = principal_amount * apy * duration / interest_denominator;
```

Interest is therefore `0` whenever `principal_amount * apy * duration < 315_360_000_000`. Offer creation only enforces `apy > 0`, `principal_amount > min_principal_amount`, and a duration within bounds. It never checks that the resulting interest is non-zero, and neither does the fill path. As a result a loan can be created and serviced with a non-zero advertised APY but zero accrued interest.

Because `compute_protocol_fee` and `compute_repay_amounts` derive from `interest`, a zero-interest loan also produces a zero protocol fee, zero repay fee, and `net_principal_amount = principal_amount`. The borrower receives the full principal and repays exactly the principal: a free loan.

Example with the default config (`min_principal_amount = 10_000`, duration fixed at 3 days = `259_200`):

- principal = `10_001`, apy = `100` (1%), duration = `259_200`
- `10_001 * 100 * 259_200 = 259_225_920_000 < 315_360_000_000`, so `interest = 0`

The same truncation applies per fill on partially fillable offers: each fill slice (at least `min_fill_amount`, which is above `min_principal_amount`) computes interest on the slice, so splitting a low-APY offer into small slices can drive the interest on each slice, and thus the total, to zero even when a single full fill would have accrued interest.

RECOMMENDATION

Reject loans that would accrue zero interest. After computing `interest` in the fill path (or inside `compute_loan_amounts` / `compute_loan_amounts_with_terms`), require `interest > 0`, returning an explicit error otherwise.

RESOLUTION

Status: Acknowledged.

LOW **L-10** CONTINUED 2/2

ACKNOWLEDGED

Loan interest can round down to zero.

— RESOLUTION

Jupiter Team — Jupiter acknowledged the issue and considers the affected loan amounts economically impractical because the account rent would exceed the principal or collateral value.

LOW **L-11**

RESOLVED

Auth rule pNFTs can brick escrow withdrawals.

CATEGORY Logical Error

LOCATION `escrow_programmable_nft_withdraw()`

REVIEW Round 1 - Main Review

DESCRIPTION

A programmable NFT can have transfer rules, like allow transfer only if this extra proof / payload is provided or allow transfer only to / from certain owners

Offerbook supports pNFT escrow, but its withdraw instruction always tells Metaplex: transfer this pNFT with `authorization_data: None`

The public withdraw entrypoint has no argument for authorization data, then the withdraw code calls `.transfer_nft(None)`, so

1. User deposits pNFT into Offerbook
2. Offerbook moves it into a token account owned by the user signer_user PDA, not the user wallet
3. Deposit also closes the user's original NFT token account in `escrow_programmable_nft_deposit.rs:152`
4. Later, only Offerbook can sign as that PDA to return the NFT
5. But Offerbook's withdraw path cannot pass the extra pNFT authorization payload
6. If the pNFT rule set requires that payload, Metaplex rejects the transfer
7. The NFT stays stuck in the PDA escrow account

RECOMMENDATION

Consider exposing authorization data on the public instruction

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <https://github.com/jup-ag/offerbook-program/pull/8/changes/088f7389c6110681a2f96222a49a2e913c89be5f>.

LOW L-12

RESOLVED

NFT misclassified as programmable deposit.

CATEGORY Validation

LOCATION escrow_programmable_nft_deposit()

REVIEW Round 1 - Main Review

DESCRIPTION

TEXT

```
1  #!account(  
2      mut,  
3      constraint = nft_metadata.key() == Metadata::find_pda(&nft_mint.key()).0  
4  )]  
5  pub nft_metadata: Account<'info, MetadataAccount>,
```

escrow_programmable_nft_deposit never checks

TEXT

```
1  is_programmable_nft_metadata(&nft_metadata)
```

So a classic Metaplex NFT can be deposited through the programmable NFT escrow path, and the program emits

TEXT

```
1  EventAsset::ProgrammableNft(EventProgrammableNftAsset {  
2      mint: ctx.accounts.nft_mint.key(),  
3      token_program: ctx.accounts.token_program.key(),  
4  })
```

even though the metadata may be `NonFungible` or `None`, not `ProgrammableNonFungible`

LOW **L-12** CONTINUED 2/2

RESOLVED

NFT misclassified as programmable deposit.

DESCRIPTION

So escrow_programmable_nft_deposit accepts any Metaplex master edition NFT with supply 1 and decimals 0, but it does not require the metadata token standard to be ProgrammableNonFungible

As a result, classic NFTs can be deposited through the programmable NFT escrow instruction and emit `EscrowDeposit` events as `EventAsset::ProgrammableNft`. This creates NFT type confusion for indexers, UIs, and any off chain accounting that treats escrow events as canonical

RECOMMENDATION

TEXT

```
1 use crate::utils::is_programmable_nft_metadata;
2
3 #[account(
4     mut,
5     constraint = nft_metadata.key() == Metadata::find_pda(&nft_mint.key()).0,
6     constraint = is_programmable_nft_metadata(&nft_metadata) @ OfferbookError::InvalidAsset,
7 )]
8 pub nft_metadata: Account<'info, MetadataAccount>,
```

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <http://github.com/jup-ag/offerbook-program/pull/8/commits/9d26056a67ea2635a9f65a7627425df5d37e3bbb>.

LOW **L-13**

ACKNOWLEDGED

Fee exemption missing for RWA collateral.

CATEGORY Logical Error

LOCATION `compute_claim_amounts()`

REVIEW Round 1 - Main Review

DESCRIPTION

The docs say if collateral is NFT / RWA, do not take the 0.1% collateral claim fee

but the program only has two meaningful claim worlds: NFT claim and token claim. If an RWA is a normal token, the code treats it as `Asset::Token`, not as RWA,

So it goes through the normal token default path and pays the fee

The program has `Token`, `ClassicNft`, `ProgrammableNft`, and `CoreNft`, but no RWA token type or fee exempt flag

NFT collateral uses a separate no fee claim path in `claim_non_fungible_loan.rs`, Fungible RWA collateral cannot reach that path

If a lender claims defaulted RWA collateral, the lender may receive slightly less collateral than promised

docs.jup.ag/user-docs/earn/offerbook/fees-and-costs (<http://docs.jup.ag/user-docs/earn/offerbook/fees-and-costs>)

RECOMMENDATION

Snapshot a fee policy at offer / loan creation, or derive it from an on chain allowlist

TEXT

```
1 pub enum CollateralFeeClass {
2     StandardToken,
3     RwaFeeExempt,
4     NftFeeExempt,
5 }
```

Then in claim settlement

TEXT

```
1 let protocol_fee = if loan.collateral_fee_class == CollateralFeeClass::StandardToken {
2     compute_protocol_fee(loan.collateral_amount, config.liquidation_fee_bps.into())
3 } else {
4     0
5 };
```

RESOLUTION

Status: Acknowledged.

INFO I-01

RESOLVED

Unbounded Max APY Configuration.

CATEGORY Validation LOCATION programs/offerbook/src/state/config.rs#L218-L222

REVIEW Round 1 - Main Review

DESCRIPTION

The `set_max_apy` configuration setter attempts to validate that the new maximum APY is non-negative, but the input type is `u32`, so the condition is always true. This means the function effectively accepts any `u32` value, including extremely large APY caps. As a result, the config authority can set an arbitrarily high APY cap. This weakens the intended protocol limit and may allow unrealistic offers with APY values that can make interest calculation fail.

RECOMMENDATION

Consider replacing the ineffective non-negative check with an explicit upper bound that reflects the maximum APY the protocol intends to support.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <https://github.com/jup-ag/offerbook-program/pull/8/changes/a3211ddd999321faacbd3aa5a88dc929cda4cce>.

INFO I-02

ACKNOWLEDGED

Config Updates Emit No Events.

CATEGORY Events LOCATION programs/offerbook/src/instructions/update_config.rs#L41-L84

REVIEW Round 1 - Main Review

DESCRIPTION

The `update_config` instruction can change admin, pause flags, fee rates, reward rates, minimum amounts, duration limits, expiry limits, and maximum APY, but it does not emit any event after applying the update. This creates an observability gap for indexers and monitoring systems that rely on program events to track and reconstruct protocol configuration changes.

RECOMMENDATION

Consider adding a dedicated config update event and emit it after each successful update.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged the finding and monitors the configuration account directly for account updates, allowing off-chain systems to observe configuration changes without dedicated events.

INFO I-03

ACKNOWLEDGED

Offers Are Not Balance Backed.

CATEGORY Unexpected Behavior

LOCATION Global

REVIEW Round 1 - Main Review

DESCRIPTION

Offer creation stores the maker's terms but does not lock, reserve, or verify the required escrow balance. Assets are only moved later when another user fills the offer. A maker can therefore publish offers without having the advertised principal or collateral in escrow. They can also withdraw escrowed assets after publishing, while the offer remains active. The offer will still appear fillable until a taker attempts to fill it and the escrow transfer fails.

Consequently, active offers can represent fake or stale liquidity, causing takers and bots to waste transactions on fills that revert and degrading the offerbook reliability.

RECOMMENDATION

Consider verifying and locking the required maker assets at offer creation, then release unused funds on cancellation, expiry, or fill.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged this as an intentional design choice. Users may create multiple offers using the same principal or collateral, while the backend filters offers that are not balance-backed.

INFO I-04

ACKNOWLEDGED

Docs deviate from market logic.

CATEGORY Documentation LOCATION <https://docs.jup.ag/user-docs/earn/offerbook>

REVIEW Round 1 - Main Review

DESCRIPTION

The public Offerbook docs describe market behavior that is not enforced by the on-chain program.

First, collateral eligibility is presented as market curated: verified Jupiter tokens, RWAs such as xStocks, and NFTs from whitelisted collections. On-chain, this is not enforced.

Second, the docs describe counter-offer acceptance as an action by the original offer creator that consumes the original offer and makes sibling counter offers no longer actionable. On-chain, filling a counter offer does not consume, cancel, or close the referenced original offer or any other pending counter offers.

Third, the docs and interface expose LTV, collateral prices, and suggested market terms. However, the program does not store or validate an LTV, price, floor, oracle result, or Jupiter Pricing API value.

These assumptions can lead to unexpected behavior and risks for integrators.

RECOMMENDATION

Update the docs and integration guidance to separate interface/indexer behavior from program guarantees.

RESOLUTION

Status: Acknowledged.

INFO I-05

RESOLVED

Counter offer UI can confuse NFT collateral.

CATEGORY Warning

LOCATION Counter offer UI

REVIEW Round 1 - Main Review

DESCRIPTION

NFT counter offers are linked through `countered_offer`, but the link does not require the counter offer's collateral asset or filter to match the referenced offer. This is expected on-chain behavior: the selected offer's own collateral terms are what matter at fill time.

However, beware of UI integration risk. A counter offer may point to Alice's original offer for a common NFT while requiring a different specific NFT or collection. If the UI presents the action as simply accepting a counter to Alice's original offer and does not clearly display the selected counter offer's actual collateral requirement, Alice may sign a fill that escrows a different NFT than she expected.

This is not an on-chain authorization bypass. The transaction still fills the selected offer and must pass that offer's collateral validation. The risk is user confusion from treating `countered_offer` as if it preserves or inherits the referenced offer's collateral terms.

RECOMMENDATION

Display the selected counter offer's actual collateral asset or filter on every accept screen and transaction confirmation. Do not infer collateral display from the referenced `countered_offer`. If counter offers are intended to modify an existing offer rather than act as independent linked offers, enforce collateral compatibility in the off-chain builder or add on-chain counter-acceptance rules.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <https://github.com/jup-ag/offerbook-program/pull/8/changes/1b1e122c5b0e8ec63654f7af93174f3ad9b96bf4>.

INFO I-06

ACKNOWLEDGED

Counters can point to expired offers.

CATEGORY Unexpected Behavior LOCATION create_token_collateral_offer.rs:74, create_token_principal_offer.rs:75

REVIEW Round 1 - Main Review

DESCRIPTION

Counter-offer creation checks the referenced offer's side, creator, status, and asset compatibility. It does not check `countered_offer.expired_at`. Since offers do not automatically change status at expiry, an expired offer can remain `Active` or `PartiallyFilled` and still pass the counter-offer constraints.

The created counter offer can be valid and fillable on its own terms. The issue is the relationship stored in `countered_offer`: it can point at an offer that is no longer fillable even though code comments and route comments describe the referenced offer as fillable.

The risk is integration drift for UIs and indexers that group counters under the referenced offer. A user may see a new counter attached to an expired offer and interpret it as an actionable modification of that offer, even though the original offer can no longer be filled.

RECOMMENDATION

If `countered_offer` is intended to reference only live opportunities, add an expiry check during counter-offer creation. If expired references are acceptable because counters are independent offers, update route and program comments so clients treat `countered_offer` as a historical relationship rather than proof that the referenced offer is currently fillable.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged the behavior and considers counters to expired offers valid because users may submit a counter after missing the opportunity to accept the original offer.

INFO I-07

RESOLVED

NFT claim event implies escrow custody.

CATEGORY Events LOCATION claim_non_fungible_loan.rs REVIEW Round 1 - Main Review

DESCRIPTION

NFT default claims transfer collateral to lender custody, then emit `EscrowDeposit` with `user: signer`, `deposit_amount: 1`, and `total_amount: 1`. For programmable NFTs the Token Metadata transfer uses the lender signer as `destination_owner`. For Core NFTs the transfer changes direct asset ownership to the lender signer. Classic NFT claims transfer to the supplied token account, which is separately covered by the classic destination report.

This differs from token-collateral claims, where the program transfers collateral into the lender user's escrow ATA before emitting `EscrowDeposit` with the actual token-account balance. An event-driven indexer or UI that treats every `EscrowDeposit` as a protocol escrow balance update can mark a claimed NFT as deposited in Offerbook escrow even though the asset was released to lender custody.

RECOMMENDATION

Use a distinct event for default claims, or include the actual destination owner and destination account in the NFT claim event. If `EscrowDeposit` is intentionally reused for custody releases, document that NFT claim events do not necessarily mean the asset is held in the lender's Offerbook escrow.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <https://github.com/jup-ag/offerbook-program/pull/8/changes/db5c112e4875664e667e9ff972497b1fbb634875>.

INFO I-08

ACKNOWLEDGED

Active Loan Fees Are Mutable.

CATEGORY Unexpected Behavior

LOCATION `programs/offerbook/src/utils.rs#L166-L198` `programs/offerbook/src/utils.rs#L208-L239`
<https://docs.jup.ag/user-docs/earn/offerbook/fees-and-costs>

REVIEW Round 1 - Main Review

DESCRIPTION

Loans snapshot the main loan terms, such as APY, duration, principal amount, collateral amount, and interest, but they do not snapshot the protocol fee rates that will be applied later during repayment or default claim. The docs state: "Protocol fees are set in the onchain configuration and can be updated by protocol admins". However, they do not clearly state that updated fees apply to already active loans.

When a borrower repays or a lender claims token collateral after default, the program recomputes the applicable protocol fee using the current `Config` values. Since the admin can update repayment, liquidation, referral, and referee fee parameters after a loan is created, the effective economics of an already active loan may differ from what users expected at fill time.

RECOMMENDATION

Clearly document that active loans remain subject to future protocol fee configuration changes or snapshot the relevant fee bps into the loan account at creation.

RESOLUTION

Status: Acknowledged.

INFO I-09

ACKNOWLEDGED

Config PDA Not Enforced.

CATEGORY Best Practices

LOCATION Global

REVIEW Round 1 - Main Review

DESCRIPTION

The protocol treats `Config` as an authority object, but most instructions only require a valid Offerbook-owned `Config` account instead of enforcing the canonical PDA. Only initialization pins `Config` to the expected seeds, while consumers trust whichever valid account is supplied. Anchor verifies ownership and discriminator for this account type, so an arbitrary user does not currently appear able to create a fake valid `Config` account. However, if another initialized account ever exists under the program (e.g. through a future update or migration), the missing PDA constraint would let callers choose that account instead of the singleton config, causing protocol logic to execute with unintended parameters.

RECOMMENDATION

Consider enforcing the canonical `Config` PDA everywhere it is consumed, not only during initialization.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged the finding and noted that the configuration account is unique because it is a PDA derived from a constant seed.

INFO I-10

ACKNOWLEDGED

Dead Collection Attribute Filter.

CATEGORY Superfluous Code

LOCATION programs/offerbook/src/instructions/non_fungible/fill_non_fungible_principal_offer.rs#L756

REVIEW Round 1 - Main Review

DESCRIPTION

`CollectionWithAttribute` is defined as an `AssetFilter` and is also exposed through generated types and events, but the collateral validation paths do not support it. For metadata NFTs it is explicitly rejected, and for Core NFTs it falls through to the default rejection branch. Since no current validation path accepts it, the filter is effectively unused dead code that exposes a non-functional protocol option.

RECOMMENDATION

Remove `CollectionWithAttribute` from the exposed protocol surface or clearly document it as unsupported until validation is implemented.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged the finding and may implement the collection-attribute filtering feature in a future update.

INFO I-11

RESOLVED

Classic NFT Loans Strand Empty ATAs.

CATEGORY Unexpected Behavior

LOCATION `programs/offerbook/src/instructions/non_fungible/fill_non_fungible_principal_offer.rs#L396-L410`
`programs/offerbook/src/instructions/non_fungib...`

REVIEW Round 1 - Main Review

DESCRIPTION

There is an escrow lifecycle asymmetry between programmable NFTs and classic NFTs. For pNFTs, after the NFT is transferred from the borrower escrow ATA into the loan vault, the program closes the emptied borrower escrow account. For classic NFTs, the NFT is also moved into the loan vault, but the empty protocol-controlled borrower escrow ATA is left open. Keeping the account open can make repayment simpler because the collateral can be returned to an existing ATA. However, if the loan defaults, the NFT is claimed by the lender and the borrower escrow ATA remains empty and unused.

RECOMMENDATION

Consider closing the classic NFT borrower escrow ATA after the collateral is locked, mirroring the pNFT flow.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <https://github.com/jup-ag/offerbook-program/pull/8/changes/26af3b74bfd3490a586ebe95ecbd76134c89e4ab>.

INFO I-12

ACKNOWLEDGED

Config min hikes strand live offers.

CATEGORY Warning LOCATION fill_token_principal_offer.rs, fill_token_collateral_offer.rs

REVIEW Round 1 - Main Review

DESCRIPTION

The fungible fill paths re-validate the computed amount against the current `Config` minimums, not the values that were in force when the offer was created. Offers only check `> config.min_*` at creation time and then store their fixed terms.

An admin who later raises `min_collateral_amount` or `min_principal_amount` above an existing offer's fill amount turns that offer into a permanent revert. The offer remains `Active` or `PartiallyFilled` in the book, but every fill attempt now fails with `InvalidFillAmount`. For a partially-fillable offer that has already taken some fills, this bricks the remaining capacity in place.

RECOMMENDATION

Snapshot the relevant minimums onto the offer at creation and validate fills against the snapshot, or explicitly document that tightening `min_principal_amount` / `min_collateral_amount` retroactively disables live offers.

RESOLUTION

Status: Acknowledged.

INFO I-13

RESOLVED

Misdocumented Repay Fee.

CATEGORY Documentation

LOCATION programs/offerbook/src/state/config.rs#L63

REVIEW Round 1 - Main Review

DESCRIPTION

The default config comment states that `repay_fee_bps: 1000` is "10% of repay amount," but the implementation applies `repay_fee_bps` only to `loan.interest`. This makes the documented fee base much larger than the actual charged base and can mislead integrators and users who rely on the default config comments to model loan economics.

RECOMMENDATION

Update the comment to state that `repay_fee_bps` is applied to loan interest.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <http://github.com/jup-ag/offerbook-program/pull/8/commits/8148e85b09b432b0e23126cab4872b2a12b40bb1>.

INFO I-14

ACKNOWLEDGED

Undocumented LoanType Field.

CATEGORY Documentation LOCATION programs/offerbook/src/state/loan.rs#L25-L30

REVIEW Round 1 - Main Review

DESCRIPTION

`LoanType` is stored on every loan and emitted in events, but only fungible fill callers can choose it. NFT loans silently default to `Classic`, while fungible loans can be freely labeled either `Classic` or `Leverage` by the filler. Since `Leverage` has no documented or enforced behavior, the field is currently superfluous metadata that may confuse off-chain consumers.

RECOMMENDATION

Document `LoanType` semantics clearly, or consider rejecting `Leverage` type until leverage-specific behavior is implemented.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Closing escrow blocks classic NFT repayment.

CATEGORY DoS LOCATION fill_non_fungible_principal_offer.rs :: PrincipalClassicNftAccounts :: lock_nft

REVIEW Round 2 - Remediation Review

DESCRIPTION

The I-11 "Classic NFT Loans Strand Empty ATAs" remediation closes `borrower_collateral_escrow` after transferring a classic NFT into the loan vault. However, `RepayClassicNftAccounts` still requires the same address to deserialize as an initialized token account and does not recreate it:

RUST

```
1  #[account(  
2      mut,  
3      token::mint = nft_mint,  
4      token::token_program = collateral_token_program,  
5  )]  
6  pub borrower_collateral_escrow: Box<InterfaceAccount<'info, TokenAccount>>,
```

Anchor therefore rejects standard classic NFT repayment with `AccountNotInitialized` before the repayment handler executes. This affects both NFT offer directions and was reproduced by the Surfpool suite. Unless the borrower manually recreates the protocol controlled ATA, the loan remains active and the lender can claim the NFT after expiry even when the borrower is willing and able to repay.

RECOMMENDATION

Keep the classic NFT borrower escrow ATA open after fill, or recreate it atomically during repayment with the signer user PDA as its authority and the correct NFT mint and token program before returning the collateral.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged the finding. Its API automatically adds an instruction to create the required token account when that account does not already exist.

LOW **L-01**

ACKNOWLEDGED

Extensions bypass the emergency pause.

CATEGORY Warning LOCATION `extend_loan.rs :: extend_loan` REVIEW Round 2 - Remediation Review

DESCRIPTION

`extend_loan` uses the repayment pause check, which permits execution while the protocol is paused whenever `disable_repayment` is false:

RUST

```
1 let config = ctx.accounts.config.load()?;
2 config.fail_if_paused_and_disable_repayment()?;
```

An extension does more than repay an existing obligation. It charges a new origination fee and resets expiry to `current_timestamp + duration`. A borrower can therefore originate another loan period and keep principal and collateral committed during an incident, even though new offers, fills, and claims are blocked by the strict pause check.

RECOMMENDATION

Use `fail_if_paused()` in `extend_loan`. If extensions should remain available under selected pause conditions, introduce a separate explicit extension pause policy instead of inheriting the repayment exception.

RESOLUTION

Status: Acknowledged.

Jupiter Team — Jupiter acknowledged the finding and confirmed that the behavior is intentional.

LOW L-02

RESOLVED

Extension emits zero-amount referral rewards.

CATEGORY Rounding

LOCATION programs/offerbook/src/instructions/fungible/extend_loan.rs

REVIEW Round 2 - Remediation Review

DESCRIPTION

`extend_loan` emits referral reward events whenever the aggregate referral reward is non-zero, but it does not re-check the split reward amount before emitting the lender and borrower events. When both lender and borrower are referred, the code divides the aggregate reward by two:

RUST

```
1 let reward_amount = if both_referred {
2     open.referral_reward / 2
3 } else {
4     open.referral_reward
5 };
```

If the aggregate reward is `1`, `reward_amount` rounds down to `0`, yet both `ReferralReward` events are still emitted. The fuzzer reproduced this with an extension where the close-period referral rewards were non-zero, but the new-period origination fee was tiny enough that the split open-period reward rounded to zero. The transaction emitted two legitimate `amount: 135` referral events and two extra `amount: 0` referral events.

Event-driven indexers, analytics, or referral accounting jobs may record phantom reward events, overcount reward occurrences, and display misleading referral activity for extensions with small fees.

RECOMMENDATION

In `extend_loan`, gate each split referral reward emission on the actual `reward_amount > 0`, matching the harness oracle and the intent already used for aggregate reward checks. Apply the same pattern to both close-period and open-period referral reward emission blocks.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <https://github.com/jup-ag/offerbook-program/pull/7/changes/d1c60bd77014cc1928711acafb06d0baab90a4da>.

INFO I-01

ACKNOWLEDGED

Offer events omit extendability.

CATEGORY Events LOCATION events.rs :: OfferEventV1 REVIEW Round 2 - Remediation Review

DESCRIPTION

Offer accounts store `allow_extend`, and loans filled from those offers inherit the value. However, `OfferEventV1` and its `From<Offer>` conversion omit the field. Event based indexers and interfaces therefore cannot determine whether an active offer permits loan extensions. This hides a material term until the offer account is fetched directly or the resulting loan is created, which is particularly relevant when a lender considers a collateral offer created by a borrower.

RECOMMENDATION

Introduce a new versioned offer event containing `allow_extend` and emit it from offer creation, cancellation, and fill paths. Keep `OfferEventV1` unchanged so historical event decoding remains compatible.

RESOLUTION

Status: Acknowledged.

INFO I-02

RESOLVED

Token-2022 mutability remains.

CATEGORY Warning LOCATION `utils.rs :: validate_mint_extensions` REVIEW Round 2 - Remediation Review

DESCRIPTION

M-02 "Token-2022 mint mutability bypass" is partially fixed. The remediation revalidates Token-2022 mint extensions when an offer is filled, closing the window in which unsafe settings could be enabled between offer creation and fill. However, `validate_mint_extensions` still permits a configured transfer fee authority and transfer hook authority, and the mint is not revalidated during repayment or claim. An authority can therefore enable a fee or hook after fill, causing later loan transfers to deliver less than expected or revert.

The team intentionally accepts this residual risk to support xStocks and a wider range of Token-2022 assets.

RECOMMENDATION

Clearly document that supported Token-2022 assets may retain authorities capable of changing transfer behavior during an active loan. User facing documentation and interfaces should warn users to inspect the mint's extensions and authorities, assess the issuer or administrator controlling them, and perform their own due diligence before creating or accepting offers involving these tokens.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <https://github.com/jup-ag/offerbook-program/pull/7/changes/12dd736f2f3a4a9add19c30ff0269ee02fa205cd>.

INFO I-03

RESOLVED

Filtered counter collateral can mismatch.

CATEGORY Warning

LOCATION programs/offerbook/src/instructions/non_fungible/create_non_fungible_collateral_offer.rs

REVIEW Round 2 - Remediation Review

DESCRIPTION

I-05 "Counter offer UI can confuse NFT collateral" is partially fixed. The remediation requires matching collateral when the referenced offer identifies a concrete NFT. However, principal offers using a collection or first verified creator filter store `Asset::None` as their collateral and keep the actual restriction in `offer.filter`. The collateral counter path accepts any concrete NFT whenever the referenced collateral is `Asset::None`:

RUST

```
1 let countered_collateral = countered.load()?.collateral;
2 require!(
3     countered_collateral == Asset::None || offer.collateral == countered_collateral,
4     OfferbookError::InvalidAsset
5 );
```

The referenced filter is never checked, so a counter can point to an offer for collection A while naming an NFT from collection B. The selected counter's own collateral is still enforced at fill, so this is not an authorization bypass. The remaining risk is user and integration confusion if `countered_offer` is presented as proof that the collateral terms are compatible.

RECOMMENDATION

When the referenced offer uses `Asset::None`, validate the counter's NFT against the referenced collection or creator filter using the required metadata or asset accounts. If this cannot be enforced during creation, prevent filtered offers from being linked as compatible counters, or require clients to perform the check and clearly display the selected counter's actual collateral.

RESOLUTION

Status: Resolved.

Jupiter Team — Fixed in <https://github.com/jup-ag/offerbook-program/pull/7/changes/12dd736f2f3a4a9add19c30ff0269ee02fa205cd>.

Jupiter Team — Jupiter acknowledged the finding. The `counter_offer` field is used only to associate offers off chain, and the team does not intend to enforce counter-offer relationships in the on-chain program.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian.

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.